

CODE & COMPLEXITY Presents

# 300 ELITE DSA PROBLEMS THAT MATTER MOST

Pattern-Based Roadmap for Interview Success

15 Core Topics • 60+ Proven Patterns



# 300 ELITE PROBLEMS

---

## THAT MATTER

BY CODE AND COMPLEXITY

*Your Complete Roadmap to Interview Success*

✓ 300 Curated Problems | ✓ All Clickable Links | ✓ Easy → Hard Progression  
✓ LeetCode + GeeksforGeeks | ✓ 15 Major Topics | ✓ 78 Patterns | ✓ 2-Month Structured Plan

### A Message Before You Begin

*"The expert in anything was once a beginner."*

— Helen Hayes

### Why This Roadmap Works

Most resources overwhelm you with 500+ random problems. This roadmap is different. I know -- everyone says that. So instead of just claiming it, here's proof :

Every problem here has been:

- ✓ Verified to appear in real interviews at FAANG and big tech companies
- ✓ Selected for maximum pattern coverage with minimum redundancy
- ✓ Ordered by increasing difficulty for natural progression
- ✓ Linked directly to LeetCode or GeeksforGeeks
- ✓ Organized by proven problem-solving patterns, not random topics

### How to Use This Roadmap

- ✦ Click any problem name to open it directly in your browser
- ✦ Follow the 2-month plan (at the end) for structured preparation
- ✦ Always start with easy problems before moving to hard
- ✦ Understand the PATTERN, not just memorize solutions
- ✦ Time yourself: 15 min (easy), 30 min (medium), 45 min (hard)
- ✦ Review completed topics weekly to retain patterns

💡 **Pro Tip:** Consistency beats intensity. Solve 4-5 problems daily rather than 20 on weekends. Build the habit, and success will follow.

🔗 **You are capable of achieving greatness.**  
**Trust the process. Stay consistent. Your dream job awaits. 🚀**

# DSA ROADMAP

---

300 Problems | All Clickable Links | 2-Month Plan

## ARRAYS (33 problems)

~ Master the foundation of all data structures - arrays are tested in 90% of interviews

### 1. Basic Operations

- [Find the largest element in an array](#)
- [Find the second largest element](#)
- [Check if array is sorted](#)
- [Find missing number in array](#)
- [Remove duplicates from sorted array](#)
- [Rotate array](#)
- [Best time to buy and sell stock](#)

### 2. Two Pointers

- [Reverse array](#)
- [Move zeros to end](#)
- [Remove duplicates from sorted array](#)
- [Two sum](#)
- [Container with most water](#)
- [Three sum](#)
- [Trapping rain water](#)

### 3. Sorting

- [Merge sorted array](#)
- [Sort colors](#)

### 4. Binary Search on Arrays

- [Binary search](#)
- [Find peak element](#)
- [Search in rotated sorted array](#)
- [Median of two sorted arrays](#)

### 5. Matrix

- [Spiral matrix](#)

- [Rotate image](#)
- [Set matrix zeroes](#)

## 6. Intervals

- [Merge intervals](#)
- [Insert interval](#)
- [Meeting rooms 2](#)

## 7. Prefix Sum & Subarray

- [Product of array except self](#)
- [Maximum subarray](#)
- [Subarray sum equals k](#)

## 8. Sliding Window

- [Max sum subarray of size k](#)
- [Longest substring without repeating](#)
- [Minimum window substring](#)
- [Sliding window maximum](#)

## BIT MANIPULATION (12 problems)

~ Lightning-fast operations using bitwise logic - loved by competitive programmers

### Basic

- [Number of 1 bits](#)
- [Power of two](#)
- [Count of bits](#)
- [Reverse bits](#)

### Single Number

- [Single number](#)
- [Missing number](#)
- [Single number 2](#)

### XOR & Advanced

- [Sum of two integers](#)
- [Decode XORed permutation](#)
- [Bitwise and of numbers range](#)
- [Maximum product of word lengths](#)
- [UTF-8 validation](#)



## HASHING (13 problems)

~  $O(1)$  lookups transform many  $O(n^2)$  problems into  $O(n)$  solutions

### Basic

- [Two Sum](#)
- [Contains Duplicates](#)
- [Valid Anagram](#)
- [Group anagrams](#)

### Subarray

- [Subarray sum equals k](#)
- [Longest consecutive sequence](#)
- [Continuous subarray sum](#)

### Design

- [Design HashMap](#)
- [LRU cache](#)
- [LFU cache](#)

### Advanced

- [4sum 2](#)
- [Minimum window substring](#)
- [Subarrays with k different integers](#)

## MATHEMATICS (10 problems)

~ Pure logic and math tricks - no complex data structures needed

### Basic & Number Theory

- [Palindrome number](#)
- [Reverse integer](#)
- [Sqrt\(x\)](#)
- [Pow\(x, n\)](#)
- [Count primes](#)
- [Factorial trailing zeroes](#)

### Combinatorics & Advanced

- [Pascal's triangle](#)
- [Unique paths \(math\)](#)
- [Integer to roman](#)
- [Roman to integer](#)

## STRINGS (19 problems)

~ Text manipulation is everywhere - from parsing to pattern matching

### 1. Basic Operations

- [Reverse string](#)
- [Valid palindrome](#)
- [First unique character](#)
- [Longest common prefix](#)

### 2. Two Pointers

- [Reverse words in string](#)
- [Valid palindrome 2](#)
- [Longest palindromic substring](#)

### 3. Sliding Window

- [Longest substring without repeating](#)
- [Longest repeating character replacement](#)
- [Minimum window substring](#)

### 4. Hashmap

- [Valid anagram](#)
- [Group anagrams](#)
- [Word pattern](#)

### 5. Pattern Matching

- [Implement strstr](#)
- [Edit distance](#)
- [Wildcard matching](#)

### 6. Advanced

- [String to integer](#)
- [Longest palindromic subsequence](#)
- [Palindrome partitioning](#)

## BINARY SEARCH (16 problems)

~ Master  $O(\log n)$  thinking - halve your search space repeatedly

### 1. Basic

- [Binary search](#)
- [First bad version](#)
- [Search insert position](#)

### 2. Rotated Arrays

- [Find minimum in rotated sorted array](#)

- [Search in rotated sorted array](#)

### 3. Peak & Range

- [Find peak element](#)
- [Find first and last position](#)

### 4. Matrix Search

- [Search 2d matrix](#)
- [Kth smallest in sorted matrix](#)

### 5. Allocation Problems

- [Koko eating bananas](#)
- [Capacity to ship packages](#)
- [Split array largest sum](#)
- [Book allocation](#)
- [Aggressive cows](#)

### 6. Advanced

- [Median of two sorted arrays](#)
- [Minimize max distance to gas station](#)

## RECURSION & BACKTRACKING (18 problems)

~ Break problems into smaller subproblems - essential for trees, graphs, and DFS

### 1. Basic Recursion

- [Factorial](#)
- [Fibonacci](#)
- [Reverse string](#)
- [Power\(x, n\)](#)

### 2. Divide & Conquer

- [Binary search recursion](#)
- [Merge sort](#)
- [Quick sort](#)

### 3. Subsets & Combinations

- [Subsets](#)
- [Combination sum](#)
- [Subsets 2](#)
- [Combination sum 2](#)

### 4. Permutations

- [Letter combinations](#)



- [Permutations](#)
- [Generate parentheses](#)

## 5. Backtracking

- [Palindrome partitioning](#)
- [Word search](#)
- [N queens](#)
- [Sudoku solver](#)

## LINKED LIST (17 problems)

~ Pointer manipulation mastery - understanding links is key to trees and graphs

### 1. Basic

- [Reverse linked list](#)
- [Find middle of Linked List](#)
- [Delete node in Linked List](#)
- [Remove Nth from end of Linked List](#)

### 2. Cycle Detection

- [Linked list cycle](#)
- [Linked list cycle 2](#)
- [Find Length of Linked List Cycle](#)

### 3. Reversal

- [Reverse linked list 2](#)
- [Reorder list](#)
- [Reverse nodes in k groups](#)

### 4. Merge & Manipulation

- [Merge two sorted lists](#)
- [Partition list](#)
- [Sort list](#)
- [Merge k sorted lists](#)

### 5. Advanced

- [Intersection of two linked lists](#)
- [Copy list with random pointer](#)
- [Flatten multilevel doubly linked list](#)

## STACKS & QUEUES (19 problems)

~ LIFO/FIFO structures enable elegant solutions for nested and sequential problems

### 1. Design

- [Implement stack using queues](#)
- [Implement queue using stacks](#)
- [Min stack](#)

### 2. Basic Operations

- [Valid parentheses](#)
- [Backspace string compare](#)
- [Remove outermost parentheses](#)

### 3. Monotonic Stack

- [Next greater element](#)
- [Daily temperatures](#)
- [Next greater element 2](#)
- [Online stock span](#)
- [Sum of subarray minimums](#)

### 4. Expression Evaluation

- [Evaluate reverse polish notation](#)
- [Decode string](#)
- [Basic calculator](#)

### 5. Advanced

- [Remove k digits](#)
- [Largest rectangle in histogram](#)
- [Trapping rain water](#)
- [Maximal rectangle](#)
- [Sliding window maximum](#)

## BINARY TREES (30 problems)

~ Trees are recursive structures - master them for 50% of coding interviews

### 1. Traversal

- [Inorder traversal](#)
- [Preorder traversal](#)
- [Postorder traversal](#)
- [Level order traversal](#)
- [Zigzag level order traversal](#)
- [Vertical order traversal](#)

## 2. Properties

- [Symmetric tree](#)
- [Invert binary tree](#)
- [Maximum depth](#)
- [Balanced binary tree](#)
- [Diameter of binary tree](#)

## 3. Views

- [Right side view](#)
- [Bottom view](#)
- [Top view](#)

## 4. Path & Sum

- [Path sum](#)
- [Path sum 2](#)
- [Sum root to leaf](#)
- [Binary tree maximum path sum](#)

## 5. BST

- [Search in BST](#)
- [Validate BST](#)
- [Kth smallest in BST](#)
- [Lowest common ancestor BST](#)

## 6. Construction

- [Flatten binary tree to Linked List](#)
- [Construct from preorder and postorder](#)
- [Serialize and deserialize](#)

## 7. Advanced

- [Lowest common ancestor](#)
- [Unique binary search trees](#)
- [House robber 3](#)
- [All possible full binary trees](#)
- [Binary tree cameras](#)

## HEAPS & PRIORITY QUEUES (19 problems)

~ Get min/max in  $O(1)$  and maintain order in  $O(\log n)$  - perfect for scheduling

### 1. Top-K

- [Kth largest element](#)
- [Kth largest in stream](#)
- [Top k frequent elements](#)
- [K closest points](#)

- [Kth smallest in sorted matrix](#)

## 2. Merge

- [K pairs with smallest sums](#)
- [Merge k sorted lists](#)

## 3. Scheduling

- [Meeting rooms 2](#)
- [Task scheduler](#)
- [Single threaded cpu](#)

## 4. Greedy with Heap

- [Reorganize string](#)
- [Furthest building](#)
- [Minimum cost to hire k workers](#)

## 5. Dual Heap

- [Find median from data stream](#)
- [Sliding window median](#)

## 6. Shortest Path

- [Network delay time](#)
- [Path with minimum effort](#)
- [Cheapest flights within k stops](#)
- [Swim in rising water](#)

## GRAPHS (30 problems)

*~ Model real-world networks - social connections, maps, dependencies*

### 1. Traversal

- [Flood fill](#)
- [Number of islands](#)
- [Number of connected components](#)
- [Clone graph](#)
- [Pacific atlantic water flow](#)

### 2. Cycle Detection

- [Detect cycle undirected](#)
- [Course schedule](#)
- [Detect cycle directed](#)
- [Redundant connection](#)

### 3. Shortest Path

- [Dijkstras algorithm](#)
- [Rotting oranges](#)
- [Shortest path in binary matrix](#)
- [01 matrix](#)

### 4. Dijkstra

- [Network delay time](#)
- [Path with minimum effort](#)
- [Cheapest flights within k stops](#)

### 5. Topological Sort

- [Course schedule 2](#)
- [Find eventual safe states](#)
- [Alien dictionary](#)

### 6. Union Find

- [Number of provinces](#)
- [Graph valid tree](#)
- [Accounts merge](#)
- [Number of islands 2](#)

### 7. MST

- [Min cost to connect all points](#)
- [Find critical and pseudo critical edges](#)

### 8. Bipartite

- [Is graph bipartite](#)
- [Possible bipartition](#)

### 9. Advanced

- [Critical connections](#)
- [Longest path in dag](#)
- [Strongly connected components](#)

## DYNAMIC PROGRAMMING (42 problems)

~ Optimize recursion with memoization - turn exponential into polynomial time

### 1. 1D DP

- [Climbing stairs](#)
- [Min cost climbing stairs](#)
- [House robber](#)
- [Maximum subarray](#)

- [House robber 2](#)
- [Jump game](#)
- [Decode ways](#)
- [Word break](#)

## 2. 2D DP

- [Unique paths](#)
- [Unique paths 2](#)
- [Min path sum](#)
- [Triangle](#)
- [Dungeon game](#)
- [Cherry pickup](#)

## 3. Knapsack

- [Coin change 2](#)
- [Coin change](#)
- [Subset sum](#)
- [Target sum](#)
- [Partition equal subset sum](#)

## 4. Sequence DP

- [Longest common subsequence](#)
- [Longest palindromic subsequence](#)
- [Longest increasing subsequence](#)
- [Largest divisible subset](#)
- [Edit distance](#)
- [Distinct subsequences](#)
- [Shortest common supersequence](#)

## 5. Interval DP

- [Min cost to cut stick](#)
- [Palindrome partitioning 2](#)
- [Burst balloons](#)
- [Matrix chain multiplication](#)

## 6. Tree DP

- [Unique binary search trees](#)
- [House robber 3](#)
- [Binary tree max path sum](#)
- [Binary tree cameras](#)

## 7. State Compression

- [Partition to k equal sum subsets](#)
- [Shortest path visiting all nodes](#)
- [Travelling salesman problem](#)



## 8. Advanced DP

- [Longest increasing path in matrix](#)
- [Wildcard matching](#)
- [Regular expression matching](#)
- [Russian doll envelopes](#)
- [Maximum profit in job scheduling](#)

## TRIES (8 problems)

*~Prefix trees for efficient string storage and autocomplete -  $O(L)$  operations*

### Basic & Search

- [Implement trie](#)
- [Design add and search words](#)
- [Replace words](#)
- [Longest word in dictionary](#)
- [Search suggestions system](#)
- [Word search 2](#)

### Advanced

- [Maximum xor of two numbers](#)
- [Palindrome pairs](#)

## GREEDY (14 problems)

*~ Make locally optimal choices for global optimum - prove correctness carefully*

### Basic & Intervals

- [Assign cookies](#)
- [Lemonade change](#)
- [Jump game](#)
- [Non-overlapping intervals](#)
- [Minimum arrows to burst balloons](#)

### Array Greedy

- [Jump game 2](#)
- [Partition labels](#)
- [Gas station](#)
- [Candy](#)

### Advanced

- [Task scheduler](#)
- [Queue reconstruction by height](#)
- [Course schedule 3](#)
- [Maximum profit in job scheduling](#)

- [Minimum refueling stops](#)

CODE AND COMPLEXITY

## 2-MONTH PREPARATION ROADMAP

Complete all 300 problems in 8 weeks with this structured plan:

### Week 1-2: Foundation (68 problems)

- Arrays (33 problems)
- Bit Manipulation (12 problems)
- Hashing (13 problems)
- Mathematics (10 problems)

*Target: 5 problems/day | Focus: Build fundamentals*

### Week 3-4: Strings & Searching (53 problems)

- Strings (19 problems)
- Binary Search (16 problems)
- Recursion & Backtracking (18 problems)

*Target: 4 problems/day | Focus: Pattern recognition & searching*

### Week 5-6: Linear DS & Trees (66 problems)

- Linked List (17 problems)
- Stacks & Queues (19 problems)
- Binary Trees (30 problems)

*Target: 5 problems/day | Focus: DS Mastery & Tree algorithms*

### Week 7-8: Advanced Topics (113 problems)

- Heaps (19 problems)
- Graphs (30 problems)
- Dynamic Programming (42 problems)
- Tries (8 problems)
- Greedy (14 problems)

*Target: 8 problems/day (intensive) | Focus: Advanced patterns & optimization*

### Success Tips

- Start with easy problems in each topic before moving to hard
- Understand the pattern, not just memorize solutions
- Revise completed topics weekly to retain patterns
- Time yourself: 15 min (easy), 30 min (medium), 45 min (hard)
- If stuck after 30 minutes, check hints/solution
- Join study groups or forums for discussion

---

© 2025-2026 Code and Complexity. All rights reserved.

**Licensed under Creative Commons Attribution-NonCommercial-NoDerivatives 4.0**

FOR PERSONAL USE ONLY

✗ Resale, commercial use, or redistribution prohibited

✓ Use for your own interview preparation

This resource took months to create. Please respect the work by not reselling it.

Want to use this for commercial purposes? Let's talk: [codeandcomplexity@gmail.com](mailto:codeandcomplexity@gmail.com)

---

 Follow for DSA tips, interview strategies, and detailed problem explanations

Instagram: [@codeandcomplexity](https://www.instagram.com/codeandcomplexity) | YouTube: [@codeandcomplexity](https://www.youtube.com/codeandcomplexity)